

The background image depicts a dark, atmospheric landscape under a starry night sky. Several tall, dark, rectangular towers or pillars are scattered across the terrain. Each tower is illuminated with a series of horizontal blue light bands. A central, taller tower is the focal point, with a bright, glowing blue ring of light at its top. From this central tower, several thin, curved blue lines of light extend outwards, connecting to the other towers. The ground is dark and reflective, mirroring the blue light from the towers. The overall mood is mysterious and high-tech.

SISTEMAS DISTRIBUIDOS

CLASE 1 – INTRODUCCIÓN Y RPC

Hoja de ruta

1. ¿Qué es un sistema, y cómo lo vamos a estudiar?
2. ¿De qué está hecho? Las tres abstracciones
3. ¿Qué es un sistema distribuido, y por qué queríamos uno?
4. ¿Por qué no puede ser transparente?
5. ¿Cómo se ve todo esto en un sistema real?
6. Esconder el mensaje: RPC
7. ¿Y si no me responden?

¿QUÉ ES UN SISTEMA?

Un conjunto de componentes interconectados que interactúan entre sí para producir un **comportamiento observable en su interfaz con el entorno**

Interfaz y entorno

- **Adentro** — lo que está en discusión: lo que estamos estudiando
- **La frontera** — la interfaz: API, endpoints
- **Afuera** — el entorno: lo que decidimos no discutir

El contorno declara el recorte que hicimos.

La granularidad es un punto de vista

- Cada componente puede ser, a su vez, otro **subsistema**
- El kernel contiene el file system, y este a su vez otros componentes
- Un auto es un sistema; también lo es cada uno de sus subsistemas

En un sistema distribuido **no hay mapeo directo** entre los componentes del diagrama y las máquinas físicas.

Ejemplo: Google Filesystem

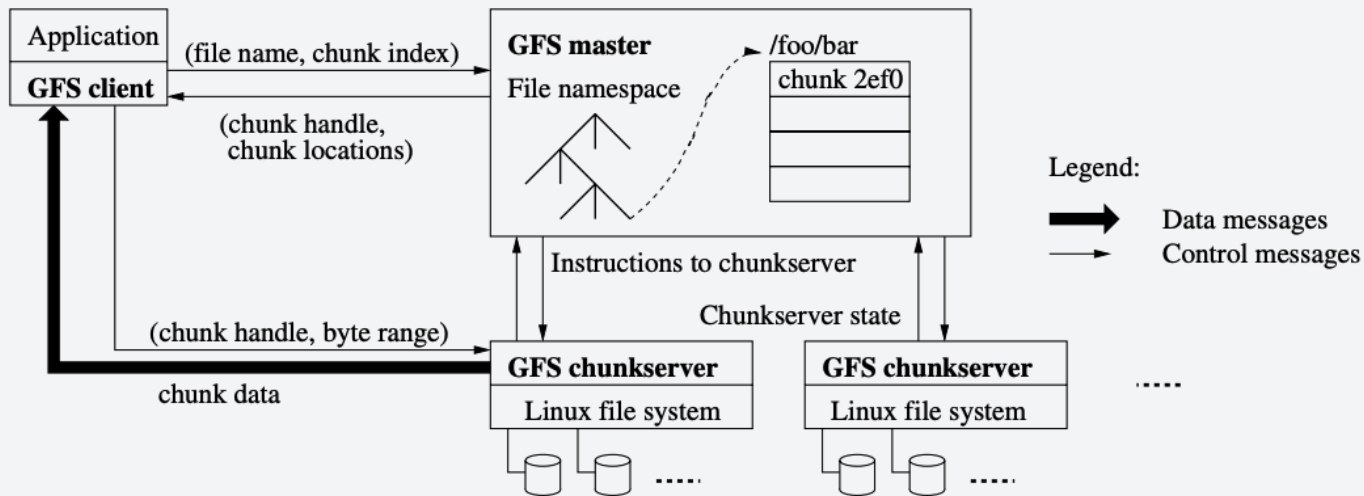


Figure 1: GFS Architecture

El master es la frontera del sistema; los chunkservers, subsistemas adentro.

¿Cómo los vamos a estudiar? Papers

- Sistemas **grandes** — la característica que los define es el tamaño, no la red
- El método: **papers de sistemas reales de empresas reales**
- Un club de lectura: los leemos y los discutimos entre todos
- Google, Amazon y Facebook son las que más innovaron
- El primero: **MapReduce**, para la próxima clase

Cuatro ejes temáticos

- **Compute** — particionar un cálculo gigante entre muchas máquinas (MapReduce)
- **Storage** — file systems distribuidos, bases relacionales y no relacionales
- **Stream processing** — datos en tiempo casi real, y mensajería
- **Cloud computing** — qué quiere decir exactamente ofrecer algo "en la nube"

Storage ocupa la mayor parte del curso: es lo más difícil de resolver.

Por qué leer papers

- **Saber leer** — y no es una obviedad: hace falta para entender, y para alimentar bien a las IA
- **Interpretar diagramas** — leerlos como sistemas, igual que el de **GFS**
- **Descubrir patrones** — el coordinador central que aparece una y otra vez
- Trabajamos con la **fuentes primaria**, no con un libro que explica el paper

Libro de apoyo de hoy: Saltzer y Kaashoek, *Principles of Computer System Design* (MIT, 2009).

LAS TRES ABSTRACCIONES FUNDAMENTALES

MEMORIA · INTÉRPRETE · ENLACE DE COMUNICACIÓN

Memoria

```
write(nombre, valor)  
valor ← read(nombre)
```

- RAM, flash, disco magnético, cinta, CD-R
- Y más arriba: RAID, file system, DBMS
- La misma interfaz, implementaciones radicalmente distintas

Intérprete

- **Referencia a instrucción** — dónde está parado
- **Repertorio** — qué operaciones sabe hacer
- **Referencia al entorno** — sobre qué estado opera

Pentium 4, UltraSPARC, el controlador de disco. Y también: Python, LISP, el bytecode de Java, JavaScript, TeX, el browser.

El intérprete abstracto

- Sólida = **control**; punteada = **información**
- Retrieve → interpret → ¿interrupción?
- Instrucción • repertorio • entorno:
las tres referencias del dibujo

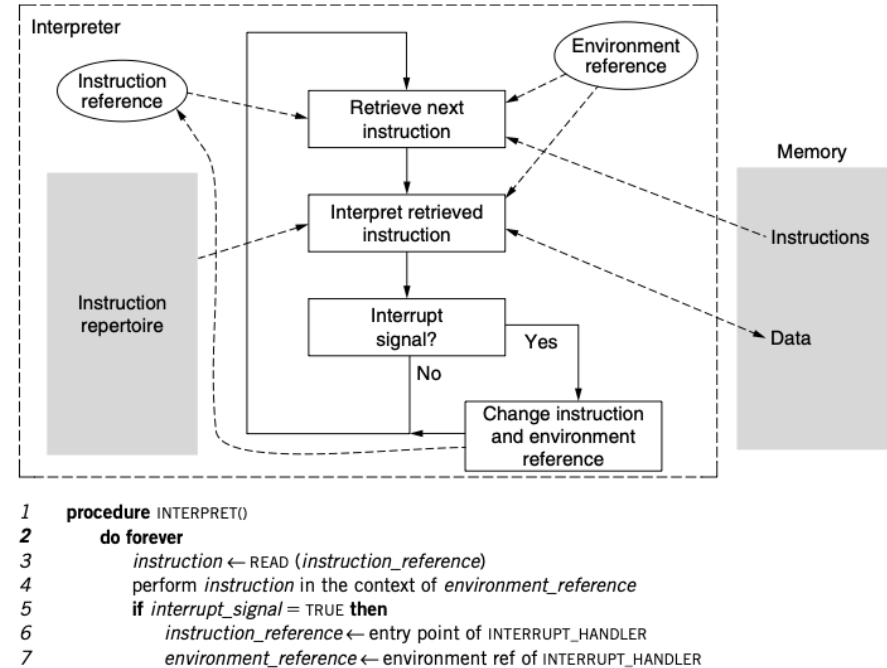


FIGURE 2.5

Structure of, and pseudocode for, an abstract interpreter. Solid arrows show control flow, and dashed arrows suggest information flow. [Sidebar 2.3](#) describes this book's conventions for expressing pseudocode.

Enlace de comunicación

```
send(nombre, outgoing_buffer)
receive(nombre, incoming_buffer)
```

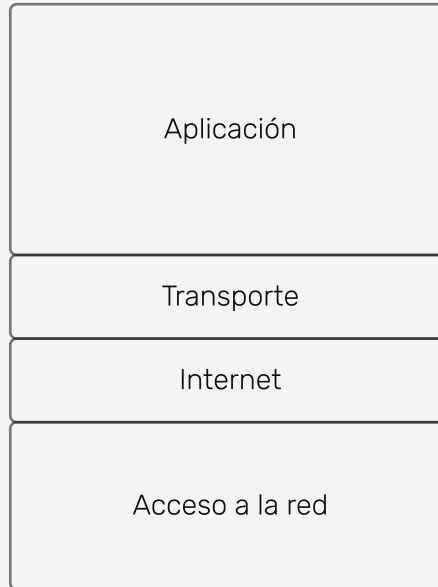
- Dos operaciones, un nombre, un dato: se parece a la memoria
- Lo que va en el buffer es el **mensaje**
- Une dos nodos que no comparten nada más
- **Pero no es memoria:** poco confiable, sin orden garantizado, el mensaje se puede perder

De lo abstracto a lo concreto: redes

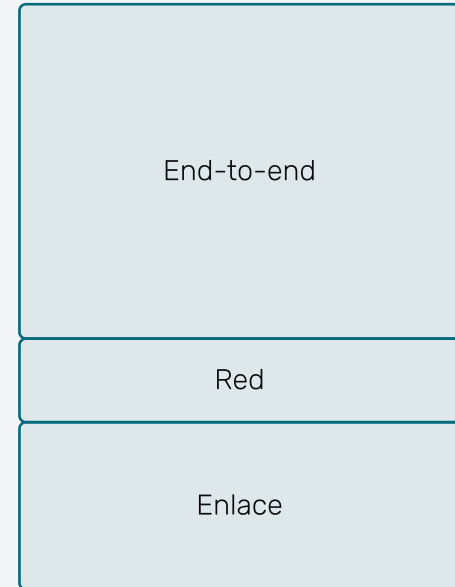
OSI



TCP/IP



La materia



Qué resuelve cada capa

- **Enlace** — mover datos de un punto físico a otro
- **Red** — desviarlos a través de puntos intermedios
- **End-to-end** — todo lo demás: una interfaz de comunicación cómoda y flexible de usar

Vamos a trabajar en la capa superior, end-to-end.

TCP o UDP: decide el diseñador

- TCP entrega un **stream de bytes** — la misma semántica que escribir un archivo
- Sobre ese stream nosotros mandamos **mensajes**, uno detrás del otro
- Los streams que vamos a estudiar son **streams de mensajes**, no los de TCP

Advertencia para toda la materia: las mismas palabras significan cosas distintas — y a veces, además, parecidas.

Sockets en C: el servidor

```
int s = socket(AF_INET, SOCK_STREAM, 0);  
bind(s, (struct sockaddr *) &addr, sizeof(addr));  
listen(s, BACKLOG);  
  
int c = accept(s, NULL, NULL);  
read(c, buf, sizeof(buf));  
close(c);
```

Seis llamadas al sistema para atender una conexión.

Sockets en C: el cliente

```
int s = socket(AF_INET, SOCK_STREAM, 0);  
connect(s, (struct sockaddr *) &addr, sizeof(addr));  
  
read(s, buf, sizeof(buf));  
close(s);
```

- `SOCK_STREAM` → TCP
- `SOCK_DGRAM` → UDP

Desde la capa de aplicación **se decide la capa de transporte.**

¿QUÉ ES UN SISTEMA DISTRIBUIDO?

Y POR QUÉ ASUMIR SU COMPLEJIDAD

Multiprocesador $\neq$ sistema distribuido

- Varios CPU sobre **una misma memoria** no es un sistema distribuido
- Lo que define al distribuido es que **la memoria no se comparte**
- Solo hay un enlace entre las partes

De lo aprendido en concurrencia, una parte se aplica acá y otra no.

¿Por qué distribuir?

- **Escalabilidad**
- **Tolerancia a fallos y confiabilidad**
- Compartir recursos
- Economía de escala
- Modularidad forzada
- Separar responsabilidades de administración

Las dos primeras son las que nos van a ocupar el cuatrimestre.

Escalabilidad

- **Vertical** — una máquina más grande: más RAM, más CPU, mejor disco
- **Horizontal** — más máquinas
- La vertical se termina: hay un tamaño máximo de máquina, y el precio se dispara antes

Tolerancia a fallos

- Componentes que fallan **independientemente** + redundancia
- Con una vida útil de **3 años** por máquina, **mil máquinas** acumulan **333 fallas por año**: casi una por día

La misma pieza que parece indestructible es, multiplicada por mil, una falla diaria.

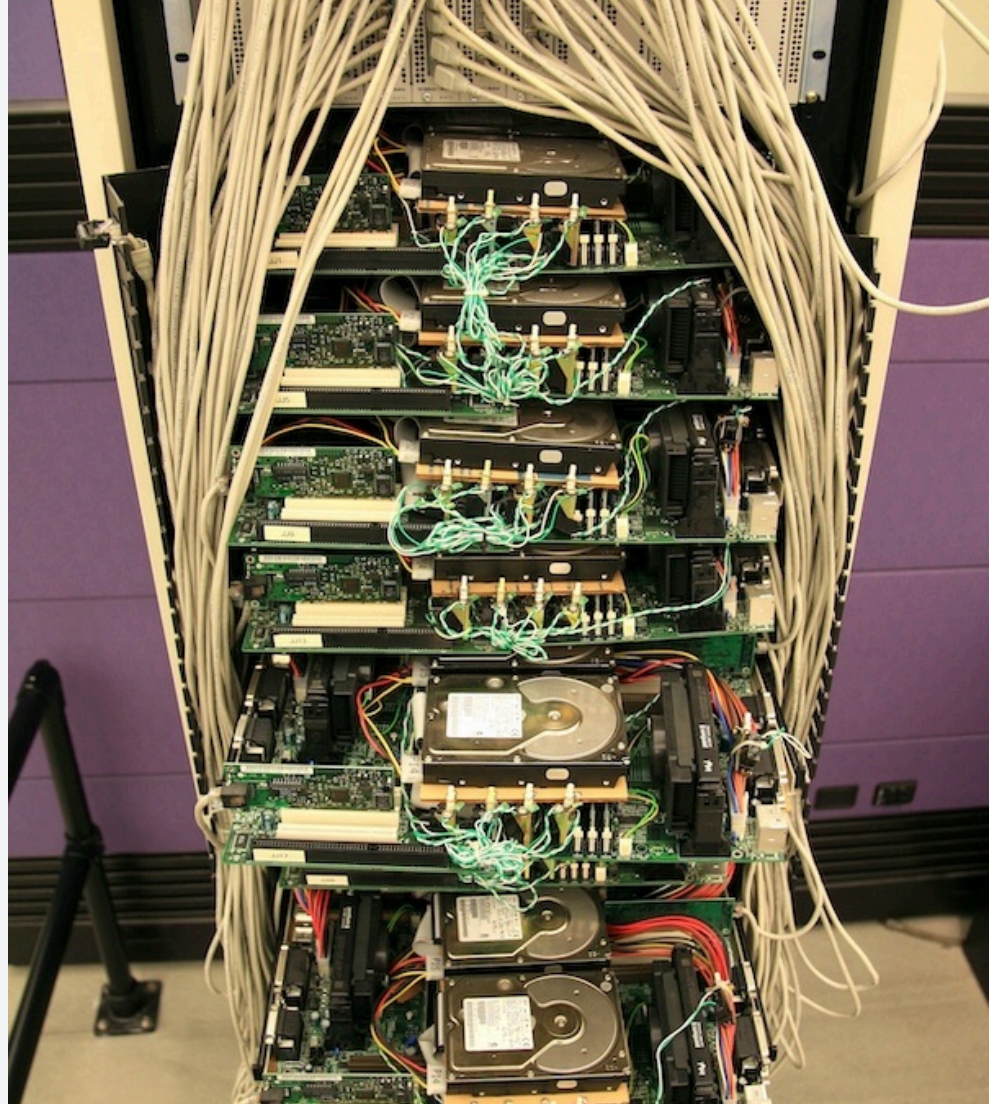
Economía de escala

- **Commodity hardware** contra mainframes
- Muchas máquinas de bajo costo, montadas en un rack, en lugar de una de gama alta
- El hardware de bajo costo falla más seguido
- Tolerar fallas deja de ser deseable y pasa a ser un requisito de diseño

Google, 1998

Placas de PC apoyadas en bandejas, atornilladas a un rack, con los cables atados a mano.

Nada de esto es hardware de servidor.





1.760 PlayStation 3

- La Fuerza Aérea de EE.UU. armó un clúster con **1.760 PlayStation 3**
- Costó unos **2 millones** de dólares; el equipamiento específico equivalente, unos **17**
- Hoy la misma idea, con GPUs, sostiene todo el entrenamiento de modelos

De ahí surge *the data center as a computer*, y de ahí Kubernetes.

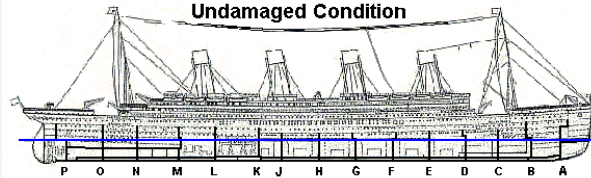
Modularidad forzada

- La modularidad que **no depende de la disciplina del programador**
- Si el módulo está en otra máquina, no se puede acceder a su memoria: solo mensajes
- Sirve para **compartimentalizar fallas**: que lo que se rompe no se propague

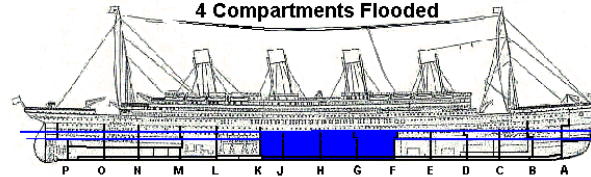
Como los compartimentos estancos del Titanic.

Permissible Flooding Conditions

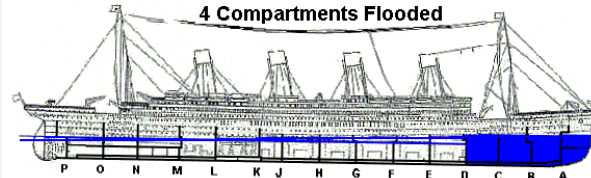
Undamaged Condition



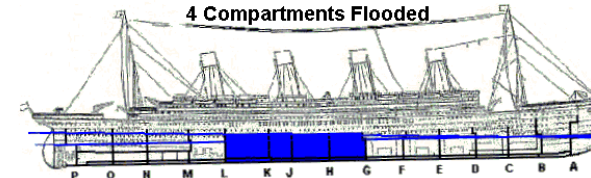
4 Compartments Flooded



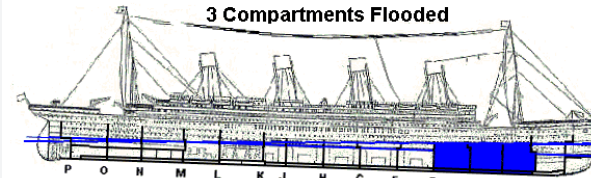
4 Compartments Flooded



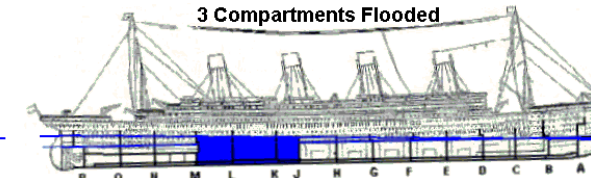
4 Compartments Flooded



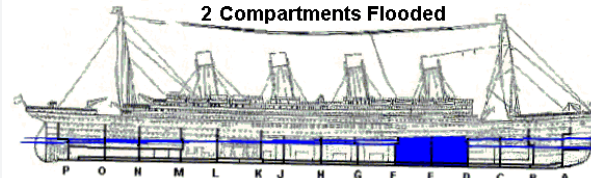
3 Compartments Flooded



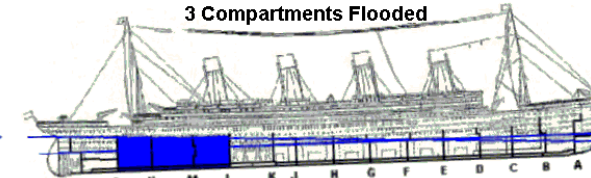
3 Compartments Flooded



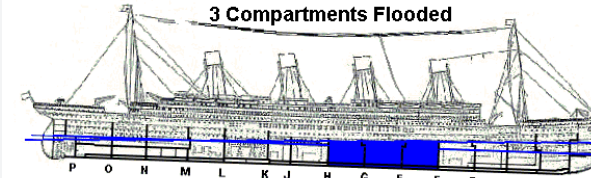
2 Compartments Flooded



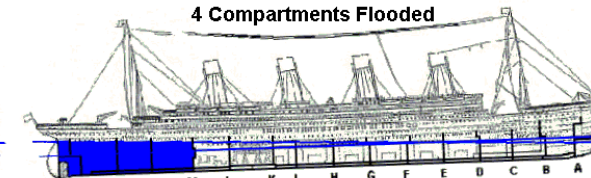
3 Compartments Flooded



3 Compartments Flooded



4 Compartments Flooded



La caída de S3 – 28/2/2017

- Un comando mal tipeado durante un mantenimiento de rutina
- Se dieron de baja **más servidores de los previstos**
- Buena parte de internet fuera de servicio, por un error humano

Summary of the Amazon S3 Service Disruption in the Northern Virginia (US-EAST-1) Region

We'd like to give you some additional information about the service disruption that occurred in the Northern Virginia (US-EAST-1) Region on the morning of February 28th, 2017. The Amazon Simple Storage Service (S3) team was debugging an issue causing the S3 billing system to progress more slowly than expected. At 9:37AM PST, an authorized S3 team member using an established playbook executed a command which was intended to remove a small number of servers for one of the S3 subsystems that is used by the S3 billing process. Unfortunately, one of the inputs to the command was entered incorrectly and a larger set of servers was removed than intended. The servers that were inadvertently removed supported

Separar responsabilidades

- Cada equipo administra su parte sin coordinar con los demás
- Las interfaces entre módulos son también interfaces entre **personas**
- El aspecto social de la ingeniería: buena parte de los microservicios se explica así

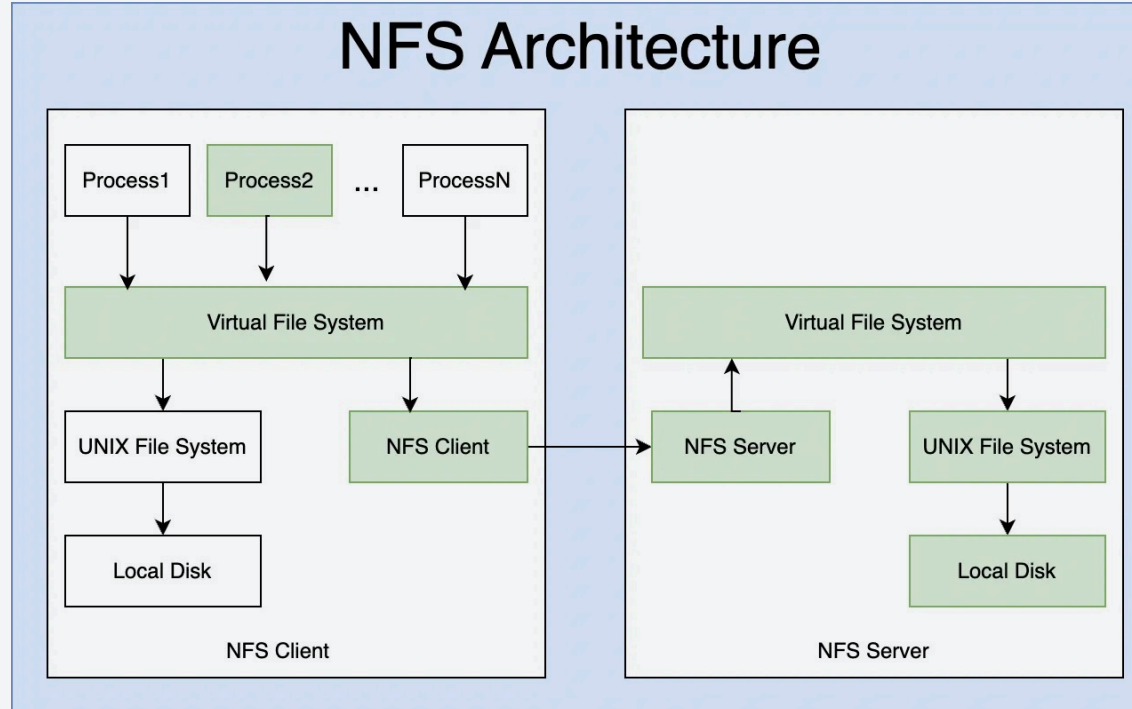
DESEABLE: TRANSPARENCIA EN LA DISTRIBUCIÓN

(pero no demasiada)

NFS: demasiada transparencia

- Un file system remoto **montado como si fuera local**, igual que ext4 o FAT
- Con la red disponible funciona bien; ante una falla, dos salidas:
- **Opción A:** devolver errores — pero `write` casi nunca falla, y nadie maneja ese caso
- **Opción B:** bloquearse hasta poder enviar — si el servidor no vuelve, indefinidamente

La arquitectura de NFS



Waldo: por qué no se puede

- **Latencia** — y sobre todo su varianza
- **Memory access** — no hay memoria compartida entre máquinas
- **Fallas parciales** — falla una parte y el resto sigue operando
- **Concurrencia** — todo pasa al mismo tiempo, en todos lados

Waldo, Wyant, Wollrath y Kendall, *A Note on Distributed Computing*, Sun Labs, 1994.

LA ORGANIZACIÓN CLIENTE- SERVIDOR

Cliente y servicio

- El **cliente** inicia el request; el **servicio** responde
- Son **roles semánticos**, no quién abrió el socket
- Alternativa: **peer-to-peer**, todos hablan con todos y no hay jerarquía
- *Servicio y servidor*: los usamos indistintamente

Raft, que veremos más adelante, es de este segundo tipo.

El servidor de tiempo: el cliente

```
1 procedure MEASURE (func)
2   SEND_MESSAGE (NameForTimeService, {"Get time", CONVERT2EXTERNAL(SECONDS)})
3   response ← RECEIVE_MESSAGE (NameForClient)
4   start ← CONVERT2INTERNAL (response)
5   func ()                                     // la función que queremos medir
6   SEND_MESSAGE (NameForTimeService, {"Get time", CONVERT2EXTERNAL(SECONDS)})
7   response ← RECEIVE_MESSAGE (NameForClient)
8   end ← CONVERT2INTERNAL (response)
9   return end - start
```


El servidor de tiempo: el servicio

```
10 procedure TIME_SERVICE ()
11   do forever
12     request ← RECEIVE_MESSAGE (NameForTimeService)
13     opcode ← GET_OPCODE (request)
14     unit ← CONVERT2INTERNAL (GET_ARGUMENT (request))
15     if opcode = "Get time" and (unit = SECONDS or unit = MINUTES) then
16       time ← CONVERT_TO_UNITS (CLOCK, unit)
17       response ← {"OK", CONVERT2EXTERNAL (time)}
18     else
19       response ← {"Bad request"}
20     SEND_MESSAGE (NameForClient, response)
```

Marshalling

- Poner los argumentos **adentro del mensaje** antes de mandarlo
- Sacar el resultado **de la respuesta** al recibirla
- Y del otro lado, lo simétrico

Este trabajo se repite en **cada** llamada. Sobre esa repetición se construye todo lo que sigue.

EL EJEMPLO REAL: UN WEB SERVER

HTTP: un cliente y un servidor reales

- **Cliente:** Chrome, Safari, Firefox
- **Servidor:** Apache, Nginx
- **Interfaz:** puerto 80, protocolo HTTP
- El mensaje tiene **headers** y **contenido**

Demo: la pestaña Network de las DevTools sobre la página de la materia.

Una interfaz bien definida

- Los errores **no se propagan** de un lado al otro
- **Aislamiento**: el servidor no puede tocar la memoria del browser
- **Simplificación**: la única interacción posible es mediante mensajes

Es modularidad forzada, ahora en un sistema de uso cotidiano.

En la realidad: escalado horizontal

- Un **load balancer** adelante: solo redirige los pedidos
- **N servidores HTTP** detrás, todos equivalentes, elegidos sin preferencia
- Funciona porque son **stateless**: no guardan nada entre requests
- El estado vive en un solo lugar: la **base de datos**

¿Dónde vive el balanceo?

- Una **máquina dedicada** adelante de todo
- Una **biblioteca** compilada dentro del cliente, que conoce a todos los servidores
- Un **sidecar**, si trabajamos con containers

Se usan las tres: la elección depende del sistema.

¿Y la base, cómo se escala?

- La base **sí** tiene estado: no alcanza con replicarla
- **Primary** (o master) para las escrituras
- **Réplicas de lectura**, que reciben la replicación y atienden los `SELECT`
- Un **backup** al costado

Sobre este problema, el del estado, se apoya buena parte de la materia.

REMOTE PROCEDURE CALL

RPC: el patrón tiene nombre

- `send` → esperar → `receive` → convertir: la misma secuencia en cada llamada
- Su estructura es la de una **llamada a un procedimiento**
- Idea de los ochenta: que llamar a algo remoto se vea como llamar a algo local
- El nombre: **RPC**, *remote procedure call*

Solución: stubs

- Un **stub** de cada lado, entre la aplicación y la red
- Esconden el marshalling y la comunicación
- La aplicación llama a una **función local**; el stub hace el resto
- Esa capa intermedia es lo que se llama **middleware**

Con stubs: el cliente

```
1 procedure MEASURE (func)
2   start ← GET_TIME (SECONDS)
3   func ()                                // la función que queremos medir
4   end ← GET_TIME (SECONDS)
5   return end - start
6
7 procedure GET_TIME (unit)                // el stub del cliente
8   SEND_MESSAGE (NameForTimeService, {"Get time", unit})
9   response ← RECEIVE_MESSAGE (NameForClient)
10  return CONVERT2INTERNAL (response)
```

Con stubs: el servicio

```
1 procedure TIME_SERVICE ()                // el stub del servicio
2   do forever
3     request ← RECEIVE_MESSAGE (NameForTimeService)
4     opcode ← GET_OPCODE (request)
5     unit ← GET_ARGUMENT (request)
6     if opcode = "Get time" and (unit = SECONDS or unit = MINUTES) then
7       response ← {"ok", GET_TIME (unit)}
8     else
9       response ← {"Bad request"}
10    SEND_MESSAGE (NameForClient, response)
```

Los stubs se generan automáticamente

```
service TimeService {  
  rpc GetTime (GetTimeRequest) returns (GetTimeResponse);  
}  
  
message GetTimeRequest {}  
  
message GetTimeResponse {  
  int64 seconds = 1;  
}
```

- El `.proto` **no es un programa**: es una especificación, y no se ejecuta
- `make gen` corre la herramienta y aparece el archivo generado: **ese archivo es el stub**
- Su primera línea advierte que no debe modificarse. Hay generador para cada lenguaje

¿Y por qué no REST?

- No es una discusión de velocidad: REST también serviría
- REST tiene **cuatro o cinco métodos**; el resto hay que inventarlo con paths
- Con RPC la operación **se diseña como una función**
- "Iniciar una máquina" en REST es un **POST** que no postea nada

REST resulta preferible cuando el cliente es un browser: no se le puede compilar un stub.

RPC no es una llamada local: la latencia

- **Más latencia.** La llamada cruza la red
- Una llamada local: nanosegundos. Una remota: milisegundos, y con varianza
- Era el objetivo inicial —que se vieran iguales— y no se cumple

RPC no es una llamada local: la falla nueva

- Una forma de falla que no existe localmente: **no response**
- ① El request **no llegó** al servidor
- ② Llegó, se ejecutó, y **no volvió** la respuesta

Desde el cliente, los dos casos son **exactamente iguales**: silencio.

¿Y SI NO ME RESPONDEN?

LAS TRES SEMÁNTICAS DE ENTREGA

At-least-once

- **Timer + retry**: si no llega respuesta, se reenvía, tantas veces como haga falta
- Si el stub responde **OK** → el servidor recibió el mensaje **al menos una vez**
- Si el stub responde **error** → no sabemos nada
- Exige que la operación del servicio sea **idempotente**

Idempotente y no idempotente

- **Borrar un archivo** — borrarlo cinco veces deja lo mismo que borrarlo una
- **Transferir dinero** — cinco reintentos de \$200.000 acreditan un millón
- La diferencia no está en la red: está en **qué hace la operación**

At-least-once no es una propiedad del transporte: es un contrato con el servicio.

At-most-once

- **No hay retry.** Es el default de gRPC
- Si el stub responde **OK** → el servidor lo recibió **exactamente una vez**
- Si responde **error** → no sabemos
- Más difícil de lo que parece: **la red puede duplicar mensajes** por sí misma

Por eso el stub se apoya en TCP y no en UDP.

Exactly-once

- Si responde **OK** → llegó, y llegó una sola vez
- Si responde **error** → no llegó **ninguna** vez
- El caso ideal. Y por lo tanto: **imposible de garantizar**

Basta con que el servidor ejecute la operación y después se destruya: la respuesta no llegará nunca.

La aproximación: idempotency ID

```
ADD_PLATA(suma, idempotency_id)
```

- El cliente manda **at-least-once**, con un identificador único por operación
- El servidor **persiste** los IDs que ya procesó
- Si llega un ID repetido, no lo ejecuta: devuelve la respuesta anterior
- Una operación no idempotente se vuelve idempotente **por construcción**

Las tres semánticas, comparadas

	Retry	Si responde OK	Le exige al servicio
At-least-once	sí	al menos 1 vez	operaciones idempotentes
At-most-once	no	exactamente 1 vez	nada, pero necesita TCP
Exactly-once	sí + ID	exactamente 1 vez	IDs + persistencia (y aun así, aproximado)

Si responde **error**, en los dos primeros casos: no sabemos.

Para la próxima

MapReduce — el paper de Google

Leerlo antes de la clase